Behaviour-Driven Development (BDD), Gherkin, and Testable Acceptance Criteria

Software Engineering & Project Management • Student Learning Book

Learning outcomes

• Explain Behaviour-Driven Development (BDD) and its relationship to requirements and testing.

• Distinguish examples, acceptance criteria, test cases, and automated checks.

• Write clear Given–When–Then scenarios using Gherkin-style syntax.

• Identify good and weak scenarios and improve them.

• Connect user stories, acceptance criteria, scenarios, defects, and Jira work items.

• Apply BDD to a College Event Management System (CEMS).

Running case study: Students use CEMS to browse college events and register when seats are available. The system must prevent duplicate registration and overbooking.

1. What Is Behaviour-Driven Development?

Behaviour-Driven Development (BDD) is a collaborative approach to describing and checking software behaviour through concrete examples that stakeholders, developers, and testers can understand. Instead of discussing a requirement only in abstract terms, the team explores examples of how the system should behave in particular situations.

BDD helps bridge the gap between business language and technical implementation. A well-written scenario makes expected behaviour explicit and can guide manual testing or, when supported by suitable automation tools, become an automated executable check.

BDD is not simply writing test scripts after coding. Its greatest value comes from discussing examples early—before implementation—so that misunderstandings are found while they are still inexpensive to correct.

2. BDD, TDD, and Traditional Testing

Approach	Main emphasis	Illustration
BDD	Shared understanding of desired behaviour using examples	When an event is full, a student cannot register
TDD (Test-Driven Development)	Developers write a failing automated test, implement code, then refactor	Write a unit test for the capacity-check function before implementing it
Traditional test execution	Checks a built system against expected results; may be manual or automated	Run a test case against the registration page

These approaches can complement one another. BDD scenarios often describe externally visible behaviour; TDD may be used inside the implementation to guide small design decisions. Neither approach removes the need for good test data, risk analysis, exploratory testing, or quality review.

3. From Requirement to Scenario

Consider this broad requirement: “The system must manage event registration correctly.” It is too vague to test directly. The team should clarify what “correctly” means and identify examples.

1. Identify the user goal: a student wants a place at an event.

2. Clarify important rules: the event must be published, seats must remain, and duplicate registration is not allowed.

3. Agree on concrete examples with stakeholders.

4. Write scenarios using understandable language.

5. Review scenarios with the Product Owner, developers, and testers.

6. Implement the behaviour and verify each scenario manually or automatically.

A useful scenario focuses on one behaviour and has an observable outcome. Avoid including implementation details such as internal method names or database table names unless they are genuinely relevant to the behaviour under discussion.

4. Gherkin: Given, When, Then

Gherkin is a structured, readable language often used to express behaviour examples. Common keywords include Feature, Rule, Scenario, Given, When, Then, And, and But.

Keyword	Purpose	CEMS example
Feature	Names the capability being described	Event registration
Rule	States a business rule that scenarios illustrate	Only available seats can be reserved
Scenario	Names one example of behaviour	Student registers when a seat is available
Given	Describes relevant initial context	Given an event has 10 seats and 4 are reserved
When	Describes the action or event	When a student registers
Then	Describes the observable outcome	Then the student receives a confirmation

> The simple pattern is: **Given** some context, **When** an action occurs, **Then** an outcome should be observed.

5. Example Feature with Scenarios

The following is illustrative Gherkin-style text. It is readable as a specification; it becomes executable only when a test automation setup has matching step definitions and supporting code.

```
Feature: Event registration
Rule: Students can register only when seats are available

Scenario: Successful registration
Given an event is published and has available seats
And the student is signed in
When the student registers for the event
Then the registration is confirmed
And a unique registration reference is displayed

Scenario: Event is full
Given an event has no available seats
And the student is signed in
When the student attempts to register
Then the registration is not confirmed
And a message explains that the event is full
```

What makes these scenarios useful?

• They describe behaviour from a user's perspective.

• They have a clear starting context, action, and observable outcome.

• They are specific enough to guide testing without prescribing the internal code.

• They cover both a successful path and an important failure path.

6. More CEMS Scenarios

Prevent duplicate registration

```
Scenario: Student cannot register twice for the same event
Given the student already has a confirmed registration for the event
```

```
When the student attempts to register again
Then the system does not create a second registration
And the student sees a clear message
```

Event not published

```
Scenario: Student cannot register for an unpublished event
Given an event is still a draft and is not published
When the student opens the event link
Then the event is not available for public registration
```

Capacity boundary

```
Scenario: Last available seat can be reserved only once
Given an event has exactly one seat remaining
When two students attempt to register at nearly the same time
Then at most one new confirmed registration is created
And the other student is informed that no seat remains
```

> The last example is especially important because the behaviour involves concurrent requests. A test that only checks a single sequential registration may miss an overbooking defect.

7. Scenario Outlines and Examples

When the same behaviour should be checked against several data values, a Scenario Outline can reduce repetition. An Examples table supplies the values for each run.

```
Scenario Outline: Registration is rejected when an event is unavailable
Given the event status is <status>
When a student attempts to register
Then registration is <result>

Examples:
| status | result |
| draft | rejected |
| closed | rejected |
```

Use outlines when the steps and meaning remain the same but the data changes. If cases have substantially different rules, separate scenarios may be clearer.

8. Strong vs. Weak Scenarios

Weak scenario	Why it is weak	Improved direction
Given the user is valid	“Valid” is undefined	State that the student is signed in and has an active account
When the system processes the request	The user action is unclear	When the student selects Register for the event
Then it works correctly	No observable result	Then a confirmation reference is displayed and saved
Given the database has row X	Implementation detail may obscure user behaviour	Describe the relevant business state, such as available capacity
One scenario tests registration, cancellation, reports, and emails	Too many behaviours are combined	Create separate scenarios for each important outcome

9. Acceptance Criteria vs. Scenarios vs. Test Cases

Item	Purpose	Example
Acceptance criteria	Defines the conditions a backlog item must satisfy	A student cannot create duplicate registration for one event

Item	Purpose	Example
BDD scenario	Shows a concrete example of expected behaviour	Given a confirmed registration exists, when the student registers again, then no second registration is created
Test case	Gives specific setup, steps, data, and expected results	Sign in as Student A; register for Event 101; repeat; verify one registration exists
Automated check	Executes checks through code and reports results	A test runner verifies the duplicate-registration scenario

A scenario is not automatically an automated test. Automation requires implementation, data setup, reliable test environments, and maintenance. A team may also execute a scenario manually where automation is not suitable.

10. Connecting BDD with Jira

A practical traceability chain is: product requirement → Epic or Story → acceptance criteria → BDD scenarios → test evidence → defect (if behaviour fails). The exact linking features available depend on Jira configuration and any test-management add-ons.

• Write the Story around a user outcome.

• Add concise, testable acceptance criteria to the issue.

• Store detailed scenarios in the issue, a test-management tool, or a linked specification.

• Link relevant tests and defects where the team's tools support it.

• When a requirement changes, review related scenarios and tests.

• Do not mark work Done until the agreed Definition of Done and acceptance conditions are met.

Example Jira Story: “As a student, I want to register for an event with available seats so that I can reserve my place.” Acceptance criteria: successful confirmation, no duplicate registration, no overbooking, and a clear message when full. Link the BDD scenarios and testing evidence to this Story.

11. Classroom Activity: Three Amigos

The “Three Amigos” is a collaborative discussion involving business/product perspective, development perspective, and testing perspective. It is a practice rather than a mandatory Scrum event.

1. Choose one CEMS Story: student registration.

2. One participant explains the user's goal and business rules.

3. One participant asks about edge cases and implementation constraints.

4. One participant challenges assumptions and proposes test examples.

5. Together, write one happy-path scenario and at least three alternative or failure scenarios.

6. Review each Then step: can a person observe or verify the result?

7. Record unresolved questions rather than guessing at business rules.

Deliverable: one user story, acceptance criteria, four Given–When–Then scenarios, and a short list of open questions.

12. Practice Questions with Suggested Answers

1. What is BDD?

Suggested answer: A collaborative approach that uses concrete examples to clarify and verify expected software behaviour.

2. What does Given describe?

Suggested answer: The initial context or preconditions for a scenario.

3. What does When describe?

Suggested answer: The action or event being considered.

4. What does Then describe?

Suggested answer: The observable outcome that should follow.

5. Does Gherkin text automatically execute?

Suggested answer: No. It needs compatible automation tooling and step definitions to become executable.

6. How is BDD different from TDD?

Suggested answer: BDD emphasizes shared examples of system behaviour; TDD commonly guides implementation through a failing test, code, and refactoring cycle.

7. Why test both success and failure paths?

Suggested answer: To verify expected operation as well as important boundaries, invalid states, and error handling.

8. What is a Scenario Outline useful for?

Suggested answer: Checking the same behaviour with multiple sets of data.

9. Why avoid vague Then steps such as “it works”?

Suggested answer: They do not define a clear, observable result.

10. How does BDD help Jira work?

Suggested answer: It connects a story and its acceptance criteria to specific examples and testing evidence.

13. Glossary

Term	Meaning
BDD	Collaborative approach to specifying and checking behaviour through examples
Gherkin	Structured language commonly used to write readable behaviour scenarios
Given	Initial context or precondition
When	Action or event
Then	Expected observable outcome
Scenario Outline	Reusable scenario pattern populated with example data
Acceptance criteria	Conditions a work item must satisfy
TDD	Test-Driven Development; a cycle of failing test, implementation, and refactoring
Three Amigos	Collaborative discussion across product/business, development, and testing perspectives
Traceability	Ability to connect requirements, scenarios, tests, and defects

14. Homework

1. Write five Gherkin-style scenarios for a Library Management System: successful book issue, unavailable book, invalid membership, return, and overdue book.

2. For one scenario, create a manual test case with preconditions, steps, test data, and expected results.

3. Choose a CEMS Jira Story and connect its acceptance criteria to at least three scenarios.

4. Identify one concurrency or boundary condition that could cause a defect and explain how you would test it.

5. Write two examples of vague requirements and rewrite them as observable behaviour.

Next chapter: Day 28 — Project Integration and End-to-End Practical: bringing requirements, design, Git, testing, project planning, and Jira together.